Eagle Delivery

Faisal Abusara | COSC 461 | Project 1 | October 26, 2026

Part A: Planning with BFS and A*

The task is to deliver a package between two building entrances. The robot plans on a known map, then learns delivery behavior by interacting with the same environment.

Problem formulation. A state is the robot's (row, column) position. The initial state is the selected starting entrance: S, L, or H for the required tasks. Actions are UP, DOWN, LEFT, and RIGHT. In the deterministic transition model, a valid action moves one cell; a wall or boundary leaves the robot in place. The goal test checks whether the position equals the selected destination. Each attempted action costs 1, so path cost is the number of actions. Search uses valid neighboring cells because a failed move cannot improve a shortest path.

Why BFS is optimal. BFS explores routes in increasing numbers of moves. Since every move has the same cost, the first goal it removes from its queue has minimum cost.

Heuristic and implementation. Manhattan distance is h(s) = |row(s) - row(goal)| + |column(s) - column(goal)|. With no diagonal moves, the robot must cover at least these row and column differences. Walls may add steps, so the heuristic never overestimates the shortest route. A* uses f = g + h, insertion order for equal-f ties, and the supplied UP, DOWN, LEFT, RIGHT neighbor order. A strictly cheaper route replaces the stored cost and parent. Stale queue entries are skipped, and the goal is checked after removing a current entry.

Search comparison

Map	Task	BFS length	A* length	BFS expanded	A* expanded
Baseline	S-W	18	18	67	22
Baseline	L-D	22	22	79	62
Baseline	H-P	16	16	41	16
Detour	S-W	26	26	70	51
Detour	L-D	22	22	65	60
Detour	H-P	16	16	41	16

Length counts actions, so it is len(path) - 1. A node is expanded only when its successors are generated; the goal is not counted. Start = goal returns a one-state path and zero expansions. An unreachable goal returns no path.

Both algorithms found equal-length routes in all six cases. A* expanded fewer nodes in these runs, although that is not guaranteed for every map. For baseline S-W, expansions fell from 67 to 22. The closure increased the S-W route from 18 to 26 actions, while L-D and H-P kept the same shortest lengths. These results show that the heuristic can reduce search work without changing route quality.

Validation: all 14 supplied unit tests passed. Full search routes and counts are saved in results/search_routes.txt and results/search_comparison.csv.

Source files: the provided assignment and starter simulator. All measurements were generated by running this implementation; raw CSVs, per-seed summaries, commands, the prediction, and plotting code accompany the report.

Part B: Learning a delivery policy

Learning rule. The Q-table is a dictionary keyed by (state, action), with unseen values equal to 0. Epsilon-greedy selection chooses uniformly from all four actions with probability epsilon; otherwise, it chooses uniformly among the actions tied for the highest Q-value. Both choices use the supplied random generator. The update changes only the observed state-action entry:

Q(s,a) ← Q(s,a) + alpha [target - Q(s,a)]
target = reward at the goal; otherwise reward + gamma max Q(next state, next action).

The maximum includes unseen actions with value zero. Goal arrival has no continuation value. A nonterminal collection cutoff still uses the observed nonterminal update. The learner uses interaction only; it never calls BFS or A*.

Settings. For each baseline task, seeds 0, 1, and 2 each started with an empty Q-table and trained for 2,000 episodes. Alpha was 0.3, gamma 0.95, constant epsilon 0.2, and slip 0. Each episode allowed up to 200 attempted actions. After training, each table was evaluated for 100 episodes with epsilon 0 and learning frozen. This gives nine training runs and 900 evaluation episodes.

Task	Eval. success	Eval. steps	Eval. return	First 100 train success	Last 100 train steps
S-W	100%	18	22	83.0%	22.58
L-D	100%	22	18	91.3%	27.68
H-P	100%	16	24	85.0%	19.73

Values combine the three seeds equally. Evaluation steps count successful deliveries; training steps include failures. Each seed had the same final evaluation success, steps, and return, giving sample SD = 0 for those final measures.

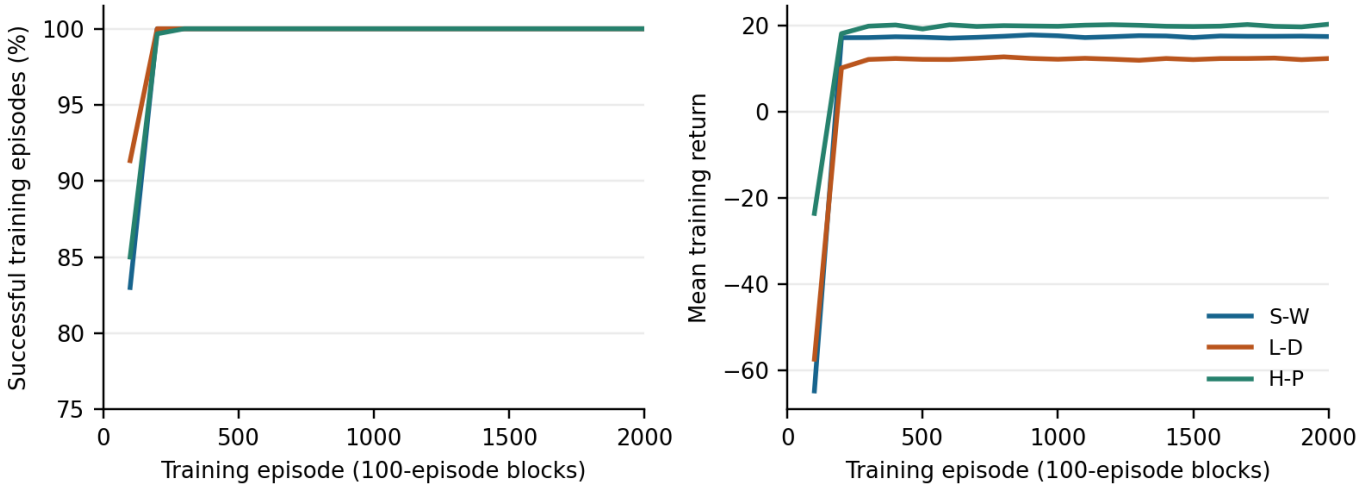


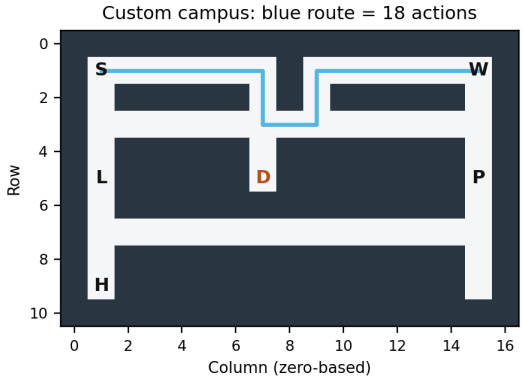
Figure 1. Training means over the three seeds in non-overlapping blocks of 100 episodes. The success axis begins at 75%. Failures remain included in both measures.

What the results show. The final learned routes matched BFS's shortest lengths: 18, 22, and 16 actions. Their returns were 22, 18, and 24, matching 40 minus delivery length. Training was less efficient because random actions continued with epsilon 0.2. For example, late S-W training averaged 22.58 actions, compared with 18 during evaluation. Mean returns improved sharply after the early episodes, but the three tasks did not have identical early success rates.

Training had 51 S-W failures, 26 L-D failures, and 46 H-P failures across 6,000 episodes per task. All were retained. The final 100 training episodes succeeded for every seed and task. Perfect final evaluation here describes these fixed, deterministic tasks; it does not establish performance on other maps or with slipping.

Part C: Custom map and exploration experiment

Question: How does exploration affect learning and final S-W delivery performance? **Prediction, recorded before the runs:** epsilon 0.05 should waste fewer actions once it learns a route; 0.5 should explore more but take longer during training. Epsilon 0.2 might offer a balance. All three may eventually learn the same shortest route when exploration is turned off.



Map design. The map is 11 by 17 cells, with a blocked border and all six building letters exactly once. All entrances are connected. Using zero-based coordinates, the main loop has corners (3,1), (3,15), (7,15), (7,1). The branch from (3,7) ends at D = (5,7). Crossing on row 3 gives an 18-action route; going via row 7 takes 26 actions. BFS and A* both found length 18, with 53 and 32 expansions.

Controlled procedure. Only constant training epsilon changed: 0.05, 0.2, or 0.5. The map and S-W task stayed fixed. Every setting used fresh tables for seeds 0, 1, 2 and the same alpha, gamma, slip, episode counts, and action limit as Part B. Evaluation again froze learning and used epsilon 0.

Training epsilon	First 100 success	Last 100 steps	Last 100 return	Training failures	Eval. success / steps
0.05	93.3%	19.00	21.00	20	100% / 18
0.20	92.7%	22.79	17.21	22	100% / 18
0.50	90.7%	35.35	4.65	28	100% / 18

Training windows average 100 episodes per seed (300 total); failures count all 6,000 training episodes per setting. Evaluation has 300 episodes per setting. All evaluation returns were 22, with zero between-seed SD.

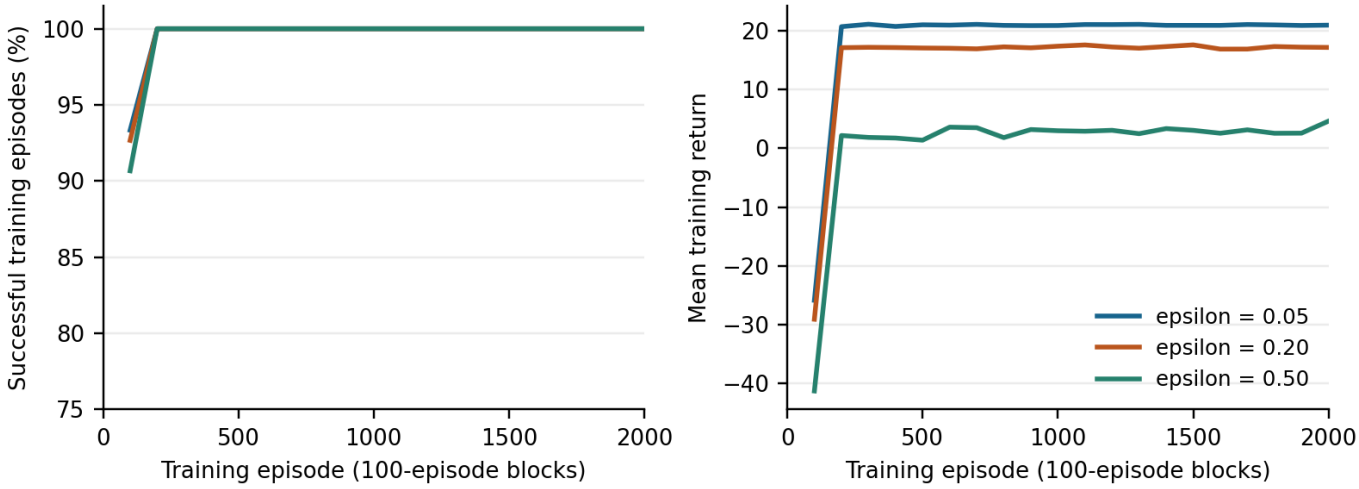


Figure 2. Custom-map training means across seeds in 100-episode blocks; the success axis begins at 75%.

Findings. The prediction about training efficiency was supported: late training averaged 19.00 actions at epsilon 0.05 and 35.35 at 0.5. Higher exploration kept causing extra actions even after a good route was learned. All three settings reached the same 18-action route in every evaluation episode. Epsilon 0.2 showed no final advantage here. The 20, 22, and 28 training failures were all kept in the results. Three seeds on one map are not enough to conclude that low exploration is always best; a larger or less familiar map could need more exploration.